

CS201 Midterm 2

17 April 2025

İlknur KARADENİZ and Olcay Taner YILDIZ

Abstract— Write only ONE function per question. Do not check the input, assume that the input is correct. Unless noted, (i) the input data structure is not empty, (ii) time complexity of the algorithm does not matter, (iii) you can not use extra data structures including arrays, (iv) you can not modify the input.

I. QUESTION (BINARY SEARCH TREE)

Write a **recursive** method in `TreeNode` Class

```
boolean hasPath(int[] path)
```

given a binary search tree (BST) and an array of integers representing a potential path. This method should return *true* if the sequence of values in the path array matches a valid path from the current node down to a node in the BST, and *false* otherwise. Hint: you should create an extra array to forward reduced path to the children.

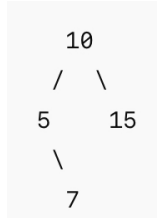


Fig. 1. Sample Tree

```
hasPath({10, 5, 7}): true
hasPath({10, 15}): true
hasPath({10, 5}): true
hasPath({10, 7}): false
hasPath({7}): false
hasPath({10}): true
```

II. QUESTION (BINARY SEARCH TREE)

Write a **non-recursive** method

```
int countEvenNodes()
```

in `Tree` class that computes and returns the number of nodes with even values in a binary search tree by using queue. You are only allowed to traverse the tree once.

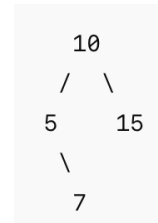


Fig. 2. Number of even nodes is 1

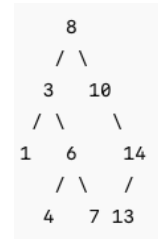


Fig. 3. Number of even nodes is 5

III. QUESTION (QUEUE)

Write a method

```
void rotateQueue(int k)
```

where k is a non-negative integer representing the number of positions to rotate the queue by, which is implemented using an array. After the rotation, the first element of the queue should move to the back k times, and the order of the other elements should shift accordingly. You are not allowed to use the `enqueue()`, `dequeue()`, or `peek()` methods. The solution should rotate the array in $O(N)$ time, where N is the number of elements in the queue. Assume that $k \leq N$.

```
[10, 20, 30, 40, 50]
```

After rotating $k = 2$ times:

```
[30, 40, 50, 10, 20]
```

```
[4, 3, 2, 7, 10, 1]
```

After rotating $k = 4$ times:

```
[10, 1, 4, 3, 2, 7]
```

IV. QUESTION (QUEUE)

Write a method

```
void reverseQueue()
```

given a queue that is implemented as a linked list. The method should reverse the order of elements in the queue without using the queue's `enqueue()`, `dequeue()`, or `peek()` methods. You must directly manipulate the underlying linked list of the queue to achieve the reversal.

```
10 → 20 → 30 → 40 → 50
```

After reverse:

```
50 → 40 → 30 → 20 → 10
```

```
4 → 3 → 7 → 2
```

After reverse:

```
2 → 7 → 3 → 4
```

V. QUESTION (HASHING)

Write a **static** method in the Hash class (array implementation)

```
boolean equalPairSums(int[] array)
```

This method should return true if any two distinct pairs in the array have the same sum. Your method should run in $O(N^2)$ time. You can use **at most one** external hash. Hint: You must store pairwise sums in the external hash table.

Warning: You are not allowed to modify the original hash class.

```
int[] list1 = {1, 4, 7, 10};
```

```
boolean result1 = Hash.equalPairSums(list1);
```

```
// result1 = true
```

```
int[] list2 = {1, 5, 10};
```

```
boolean result2 = Hash.equalPairSums(list2);
```

```
// result2 = false
```

VI. QUESTION (HASHING)

Write a **static** method in the Hash class (linked list implementation)

```
int[] difference(int[] list1, int[] list2)
```

to find the difference of the elements in two arrays and return a new array containing all elements from *list1* that are not present in *list2*. Your method should run in $O(N)$ time, where N is the total number of elements in both arrays. The difference array must contain exactly as many elements as needed (no extra or missing elements). You can use **at most one** external hash.

Warning: You are not allowed to modify the original hash class.

```
int[] list1 = {1, 2, 3, 4, 5};
```

```
int[] list2 = {2, 4};
```

```
int[] result1 = Hash.difference(list1, list2);
```

```
// result1 = [1, 3, 5]
```

```
int[] list3 = {1, 2, 3, 4, 5};
```

```
int[] list4 = {6, 10};
```

```
int[] result2 = Hash.difference(list3, list4);
```

```
// result2 = [1, 2, 3, 4, 5]
```